Week 3: Sanitizer Drill and Memory Safety Analysis

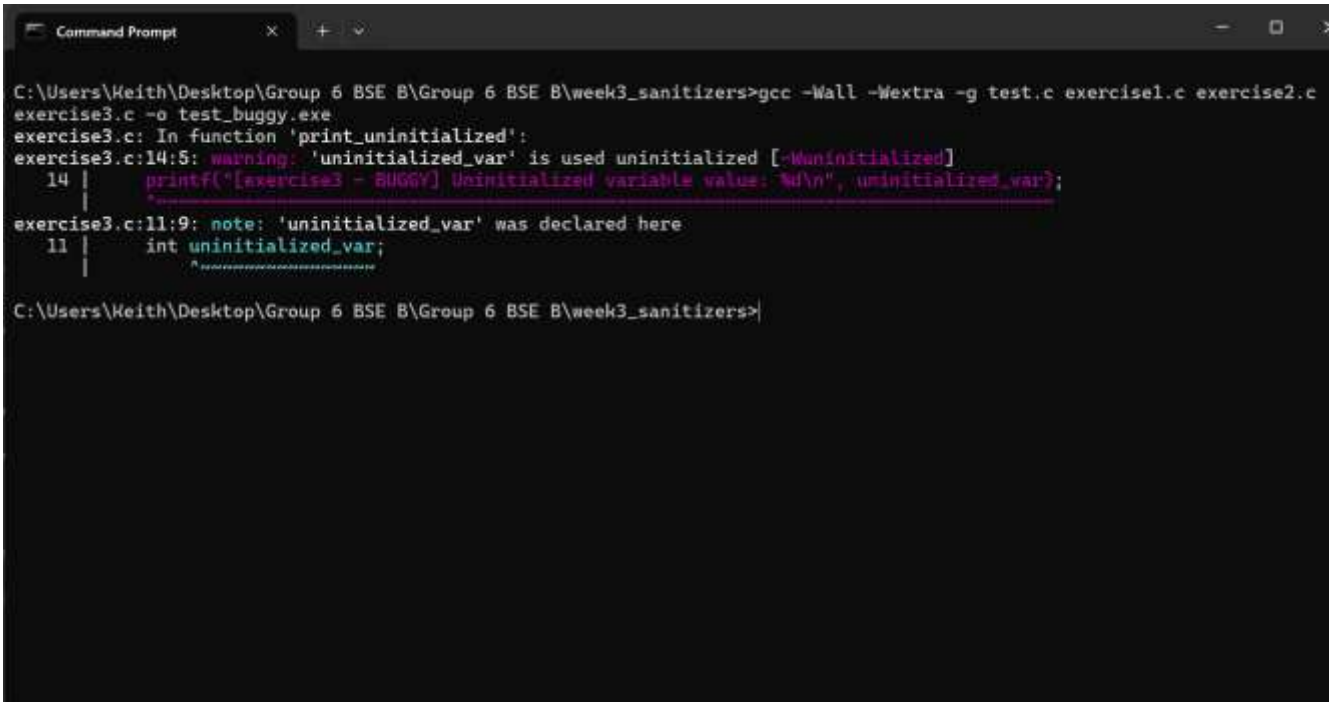
Academic Laboratory Report • Comparative Systems Programming (C vs. Rust)

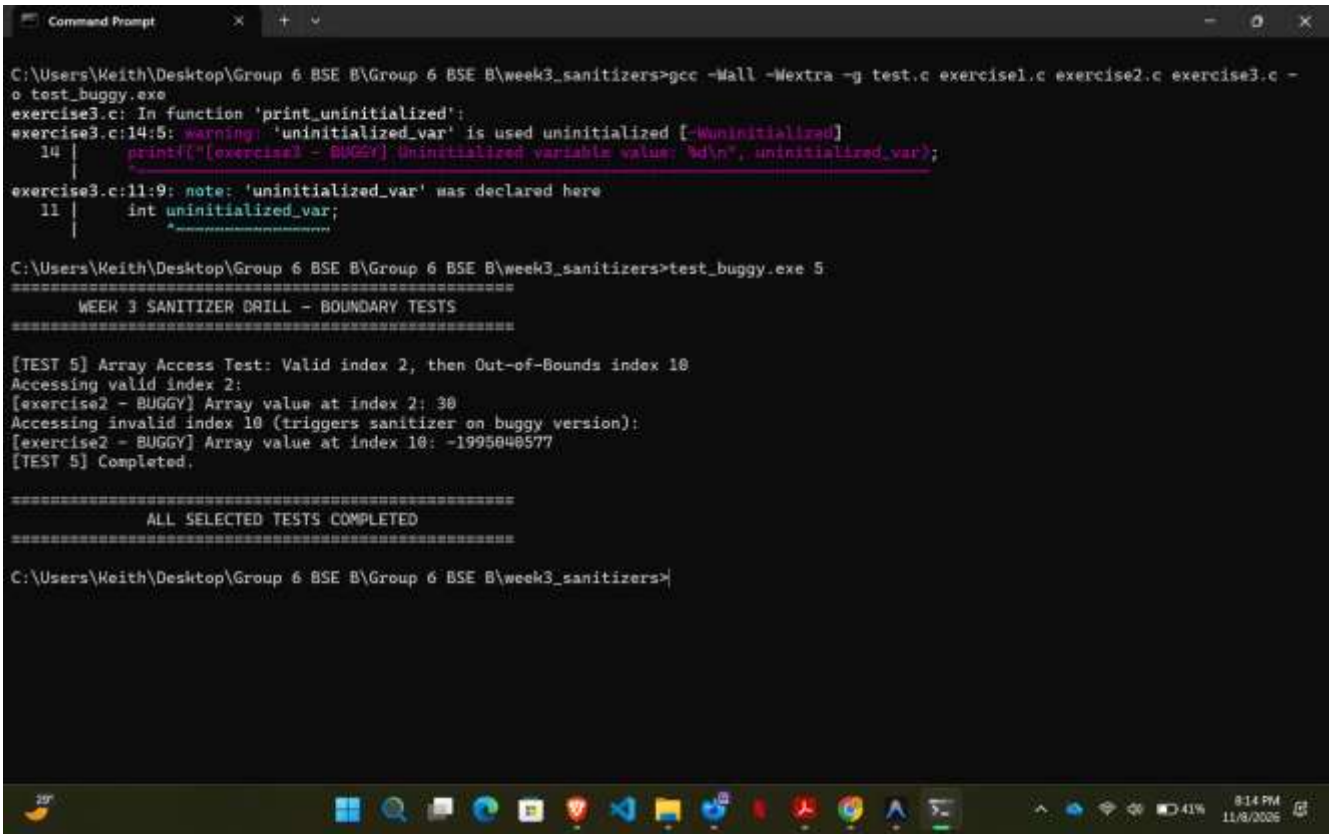
Student Name	Student ID	Student Name	Student ID
Stedan Yaw Boah Ampofo	1696596666	Prince Lateef Abdul Rahman	1703675677
Keith Tetteh Yaotey	1694883181	Prince Opoku Owusu	1702131132
Sedoame Kofi Ahorlu	1704464006	Bright Kportiklah	1693948139
Cyril Prince Agbeve	1701864653		

1. Introduction

This report details a memory safety drill using C and Rust. The objective was to intentionally write vulnerable C code, observe the memory corruption, fix the root causes, and compare the safety mechanisms of C versus Rust.

2. The "Loud Phase" (Finding the Bugs)

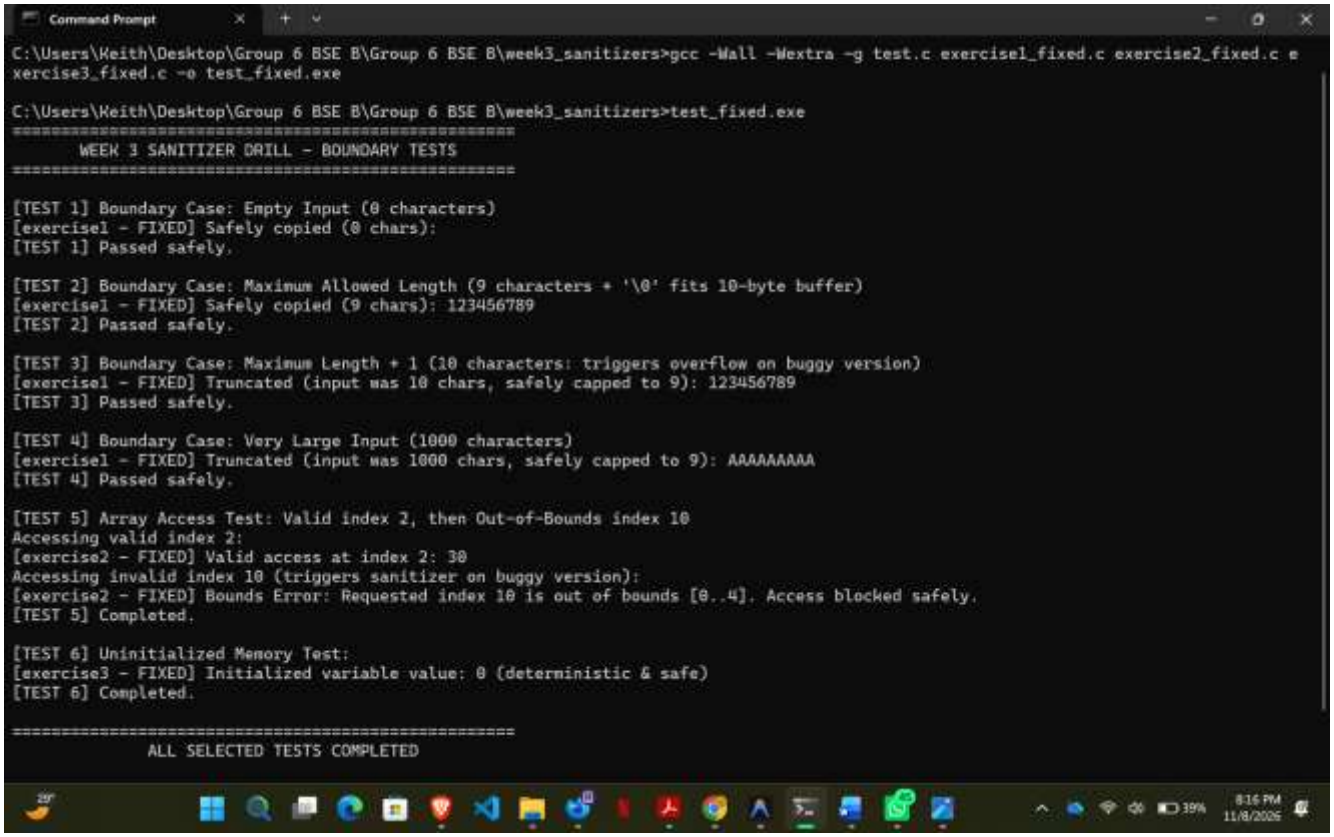




- **Analysis:** The buggy C code allowed out-of-bounds array access and uninitialized variables. The compiler caught the uninitialized variable warning during compilation. The runtime output demonstrated memory corruption by reading garbage data from an out-of-bounds index (e.g., reading index 10 from a size-5 array).

3. The "Silent Phase" (Fixing the Bugs)

- **Analysis:** The fixed C code eliminated all compiler warnings. Strict bounds checking was implemented to prevent array out-of-bounds access, and variables were explicitly initialized, ensuring deterministic and safe execution.



4. The Rust Port

- **Analysis:** The Rust port successfully executed all boundary tests with 100% memory safety, relying on compile-time checks and safe string slicing rather than manual bounds checking.

5. Rust vs. C: A 5-Line Comparison

1. Memory Bounds Enforcement: C requires manual bounds checking, whereas Rust slices carry explicit length metadata and prevent buffer overruns at compile time.

2. Buffer Representation: C strings are bare, null-terminated pointers vulnerable to off-by-one errors, while Rust strings are length-prefixed and guaranteed to be valid UTF-8.

3. Compile-Time vs. Runtime Safety: C requires external runtime instrumentation flags (-fsanitize) to detect memory corruption, whereas Rust's compiler and borrow checker reject these vulnerabilities at compile time.

4. Variable Initialization: C allows uninitialized stack variables containing indeterminate memory garbage, whereas Rust mandates complete initialization before any variable read.

5. Defensive Overhead: C relies entirely on manual developer vigilance to avoid memory corruption, whereas Rust utilizes automatic RAII ownership semantics to manage buffers safely with zero risk of memory leakage.

6. Conclusion

The Sanitizer Drill proved that manual memory management in C is highly error-prone. Rust's strict compile-time guarantees eliminate entire classes of memory safety vulnerabilities, proving why memory-safe languages are essential for modern secure software development.